Introducción a JUnit
(usando Visual Studio Code)

Estructuras de Datos - DCIC - UNS

Qué es JUnit

JUnit es una librería de Testing Unitario para Java que permite automatizar el testeo de unidades (clases) en nuestros programas.

- Produce un listado de tests indicando, para cada uno, si su ejecución fue exitosa (pass), no exitosa (fail), si hubo errores inesperados (error) o si se salteó (skip).
- Reemplaza los “testers” por línea de comandos al “estilo IPOO”, evitando la necesidad de que un humano deba examinar la salida para determinar si el objeto de testeo funciona correctamente.
- Se utiliza en esquemas de integración y despliegue continuos (CI/CD) permitiendo automatizar la compilación y despliegue de código. Ante errores, se puede detener el proceso y evitar reemplazar código que funciona por código con cambios que no lo hace.

Qué es JUnit

JUnit es una librería de Testing Unitario para Java que permite automatizar el testeo de unidades (clases) en nuestros programas.

- Está concebida para testeo unitario, esto es el testeo de una unidad de manera aislada. Para ello permite gestionar las dependencias del objeto de testeo interceptando llamadas a objetos dependientes, utilizar objetos sustitutos en reemplazo de los reales, y otras funciones que no veremos.
- También se puede utilizar para realizar otros tipos de pruebas como por ejemplo testeos de integración (comprobar que varios componentes funcionan bien integrados), end-to-end (de aplicación completa), de regresión (comprobar que lo que funcionaba no dejó de hacerlo), o de cobertura (determinando qué porcentaje de líneas de código es realmente ejecutado por los tests).

Qué es JUnit

En la materia lo utilizaremos para testear las implementaciones de los TDAs a partir de casos de prueba dados por la cátedra.

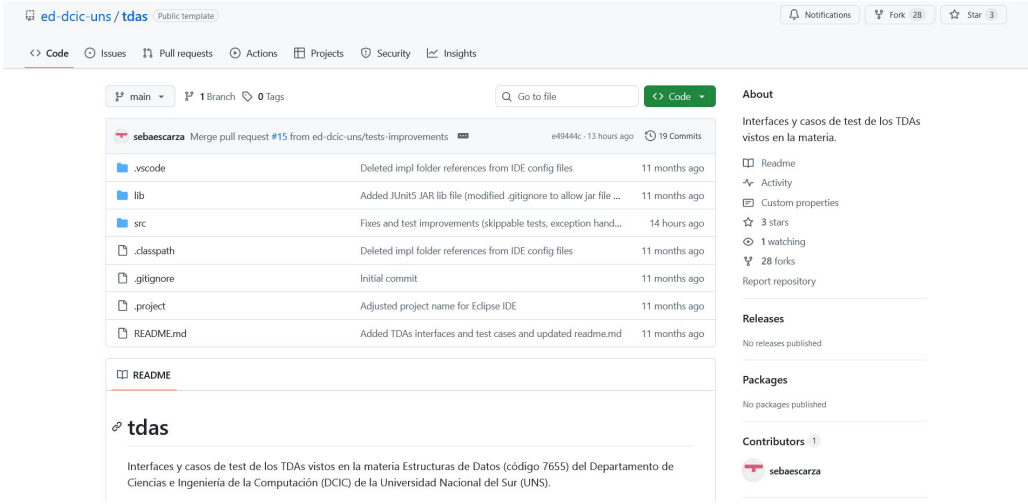
JUnit va por la versión 6. En la cátedra aún utilizamos la versión 4 por razones históricas (cuando se escribieron los casos de prueba JUnit 4 era la versión más actual). La migración implica pagar el costo de reescritura de todos los casos de prueba (las versiones modernas de JUnit modifican la sintaxis de los casos) sin obtener beneficios que lo compensen, ya que la versión 4 es perfectamente funcional a nuestros propósitos.

Obteniendo los casos de prueba para los TDAs

Obteniendo los casos de prueba para los TDAs

Los casos de prueba, al igual que las interfaces que definen los TDAs, se encuentran en el repositorio Git de la materia en Github: https://github.com/ed-dcic-uns/tdas/

Pueden obtenerse mediante alguna de las alternativas explicadas oportunamente (ver presentación correspondiente).

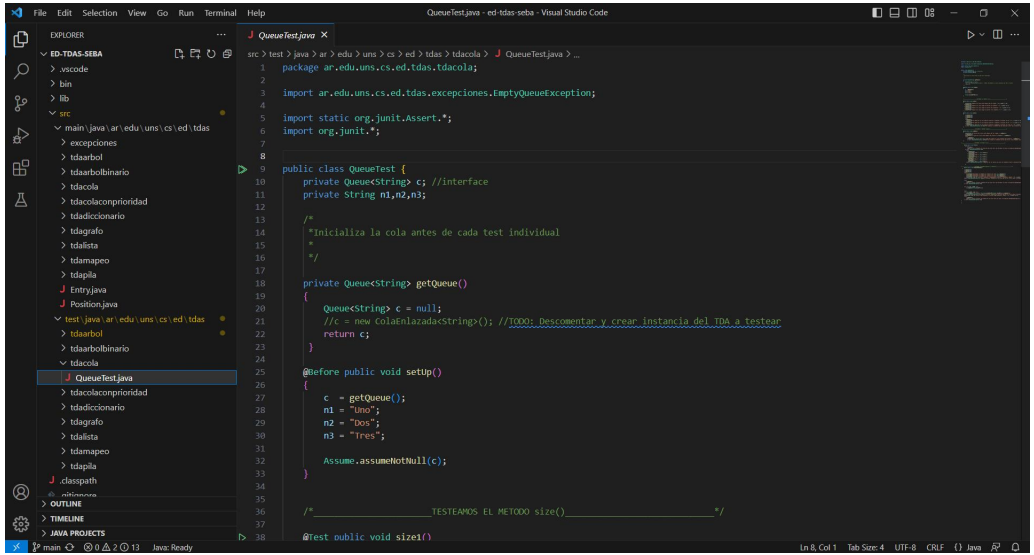


Obteniendo los casos de prueba para los TDAs

Los tests se encuentran en carpetas con ruta base: **src/test/java/...**

A partir de ahí se sigue una ruta equivalente al paquete en el que se encuentran declarados.

Ej. **QueueTest.java** en **src/test/java/ar/edu/uns/cs/ed/tdas/tdacola**



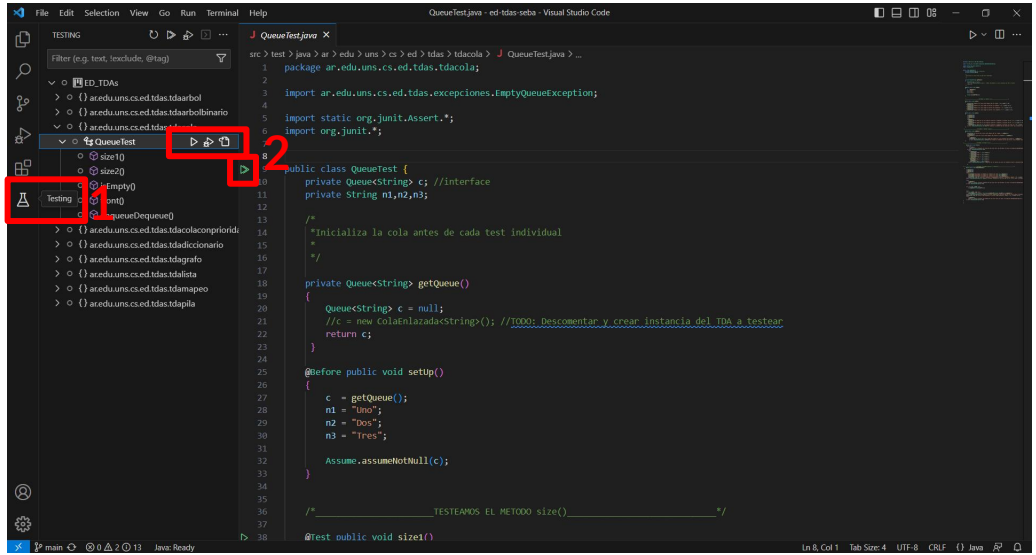
Ejecutando JUnit en VSCode

Ejecutando JUnit

En VSCode los tests se ejecutan desde el panel lateral de Testing (1).

Allí hay controles de ejecución y debug (2) para todos los tests del proyecto, para un archivo de tests o incluso para cada test individualmente.

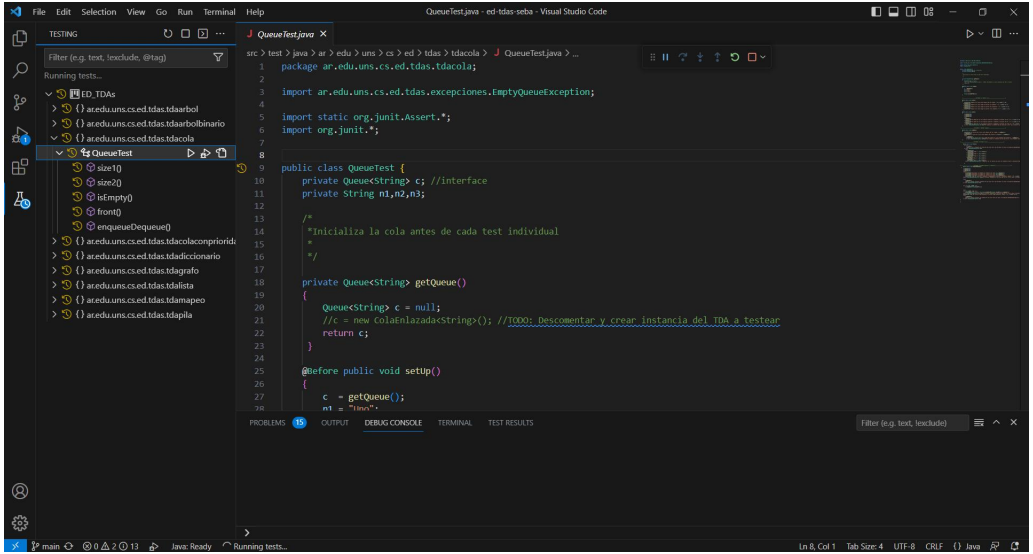
También se pueden ejecutar los tests desde los íconos (2) que VSCode agrega a la izquierda del número de línea (p/clase y métodos)



Ejecutando JUnit

El proyecto viene configurado para saltear todos los tests, ya que inicialmente no hay implementaciones de TDAs desarrolladas para testear.

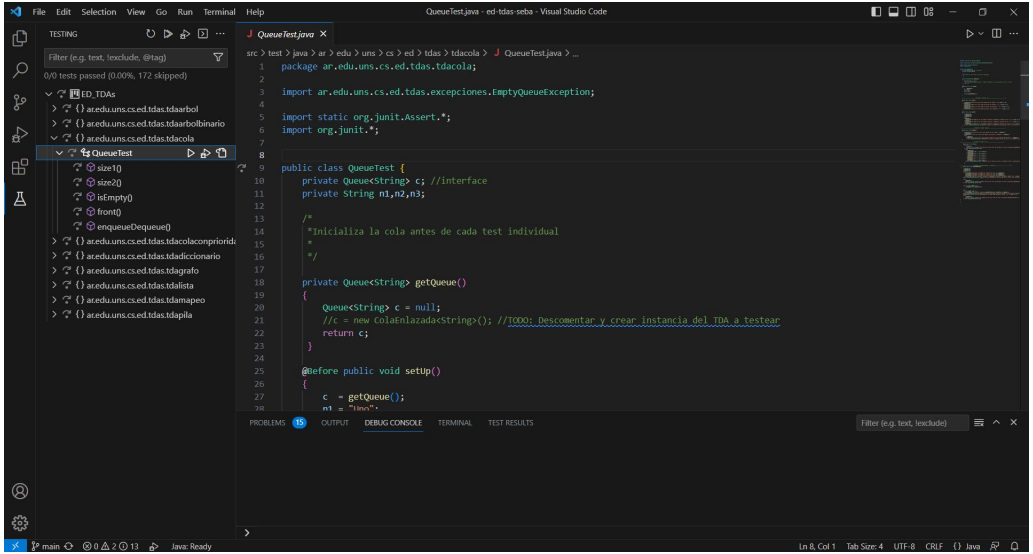
Ejecutando los tests…



Ejecutando JUnit

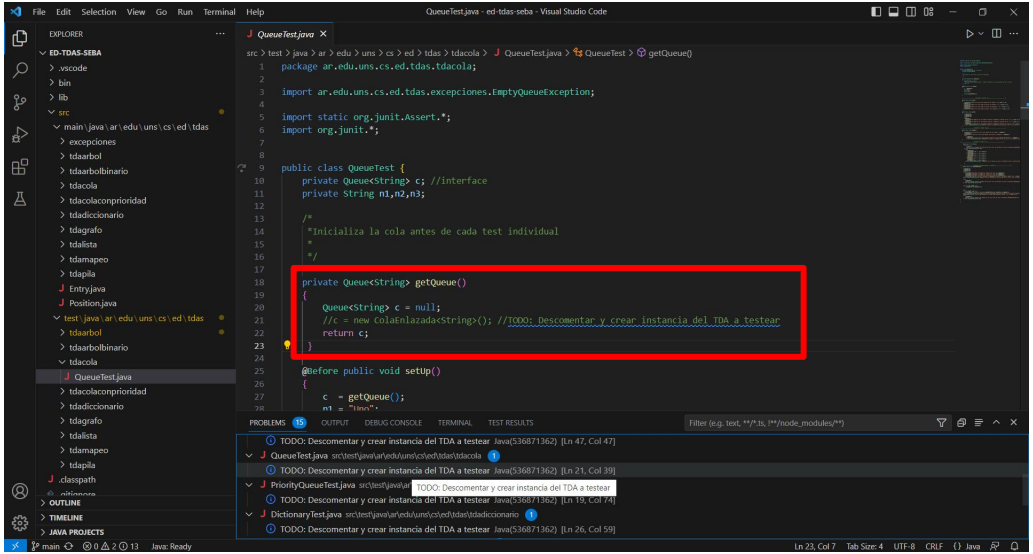
El proyecto viene configurado para saltear todos los tests, ya que inicialmente no hay implementaciones de TDAs desarrolladas para testear.

Los tests fueron salteados (skipped).



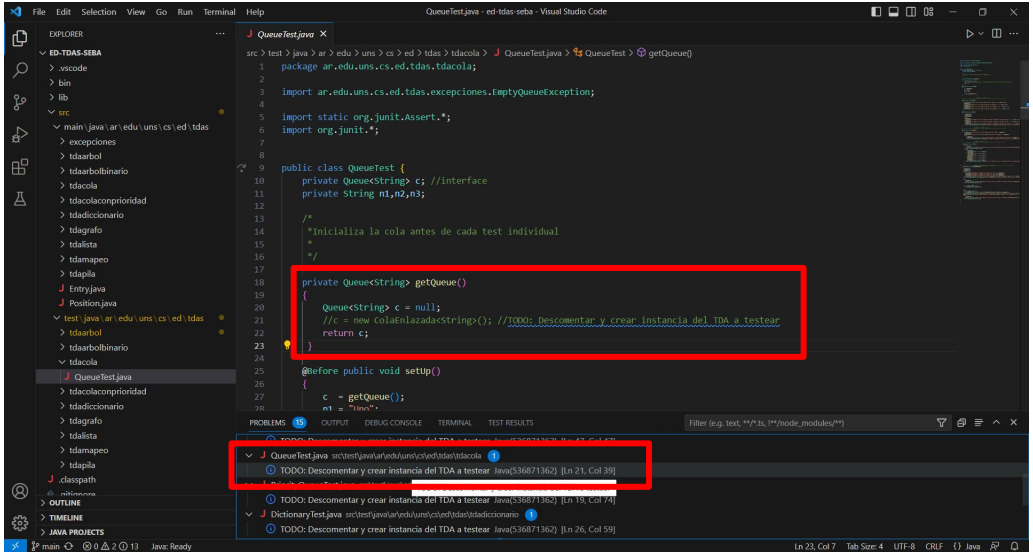
Ejecutando JUnit

Antes de utilizar los casos de prueba, se debe implementar el TDA a testear (se recomienda ubicar la implementación en la misma carpeta que la interface a implementar), y luego modificar el tester para crear una instancia a testear.



Ejecutando JUnit

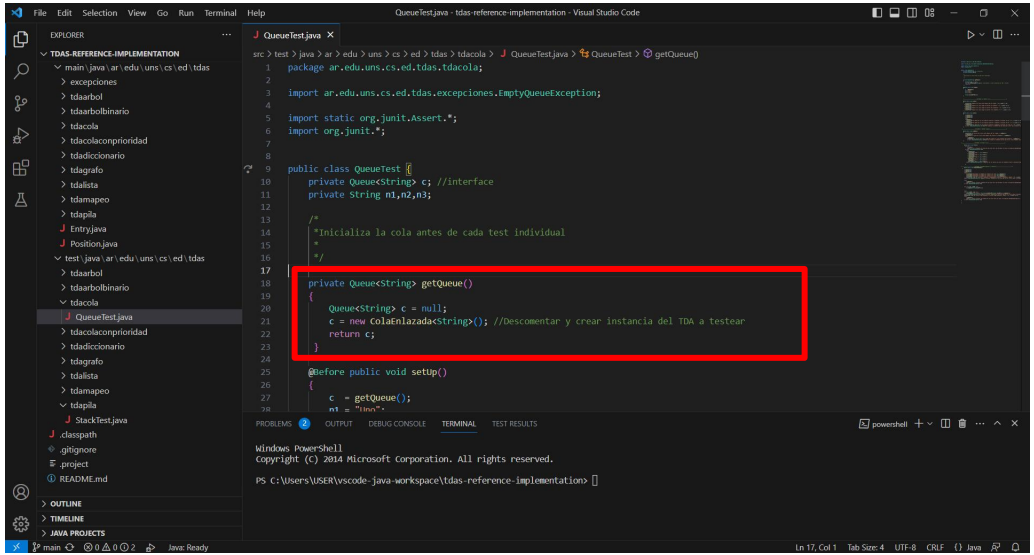
Los puntos de modificación se indican con comentarios que incluyen “TODO”. Esto hace que figuren en la lista de “problemas” a resolver en el proyecto y que sean más fácilmente localizables.



Ejecutando JUnit

Para preparar el test para su ejecución:

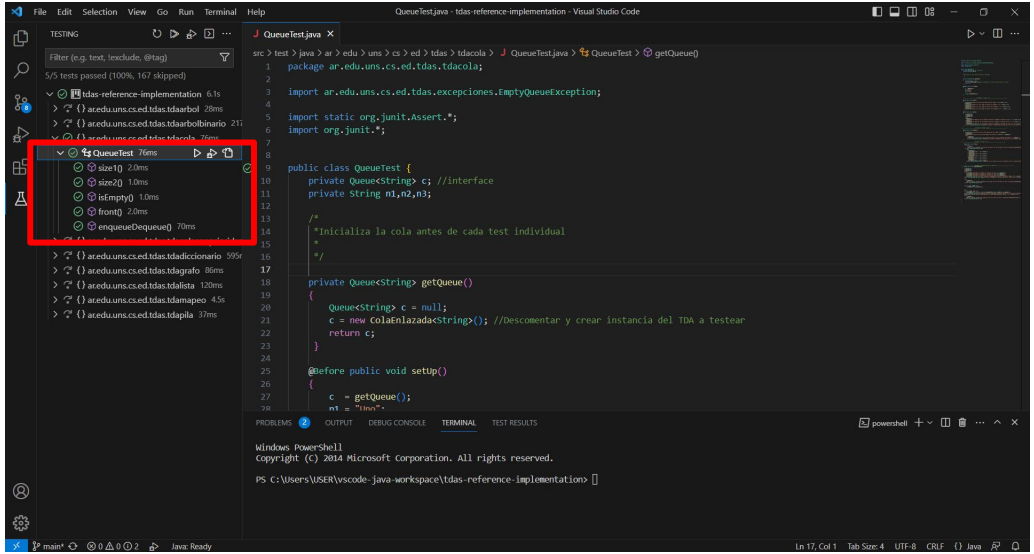
1. se descomenta la línea para crear un objeto a testear
2. se ajusta el nombre del constructor en base a la clase a testear (que compile)
3. (opcionalmente) se puede borrar el “TODO” del comentario para sacar la tarea de la lista de pendientes



Ejecutando JUnit

Si ejecutamos los tests nuevamente, ahora los tests ajustados se ejecutan correctamente (QueueTest.java para el ejemplo), mientras que los demás se siguen salteando.

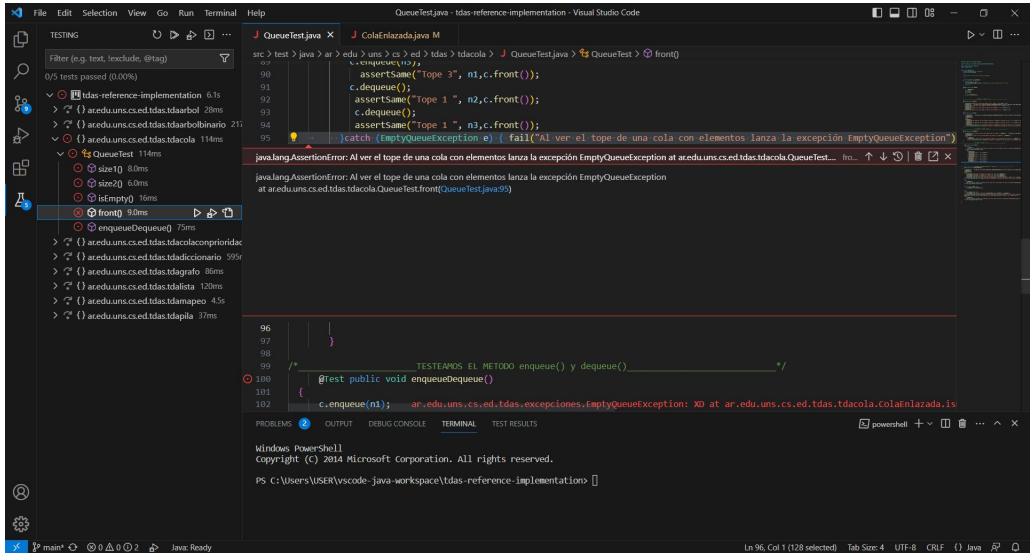
En el ejemplo se muestra que todos los tests pasaron exitosamente (tilde verde) y la implementación de cola testeada funciona correctamente.



Ejecutando JUnit

Otros resultados pueden ser:

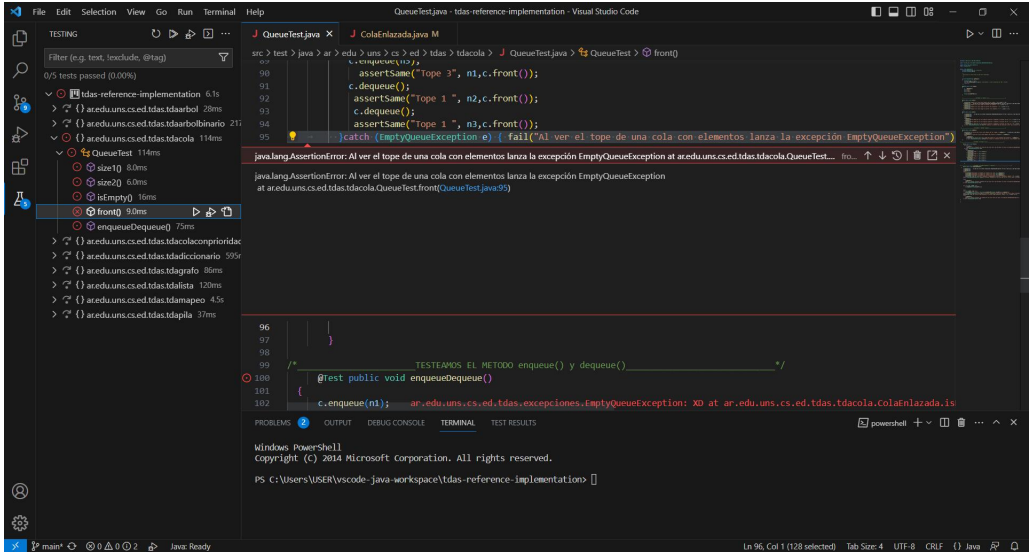
- tests fallidos (cruz roja): cuando el test completa su ejecución pero con resultados erróneos para el método testeado
- errores (punto rojo): cuando el test no completa su ejecución por errores/excepciones inesperadas



Ejecutando JUnit

Haciendo click sobre el test no exitoso se muestra un mensaje de error que es el punto inicial para depurar nuestro programa.

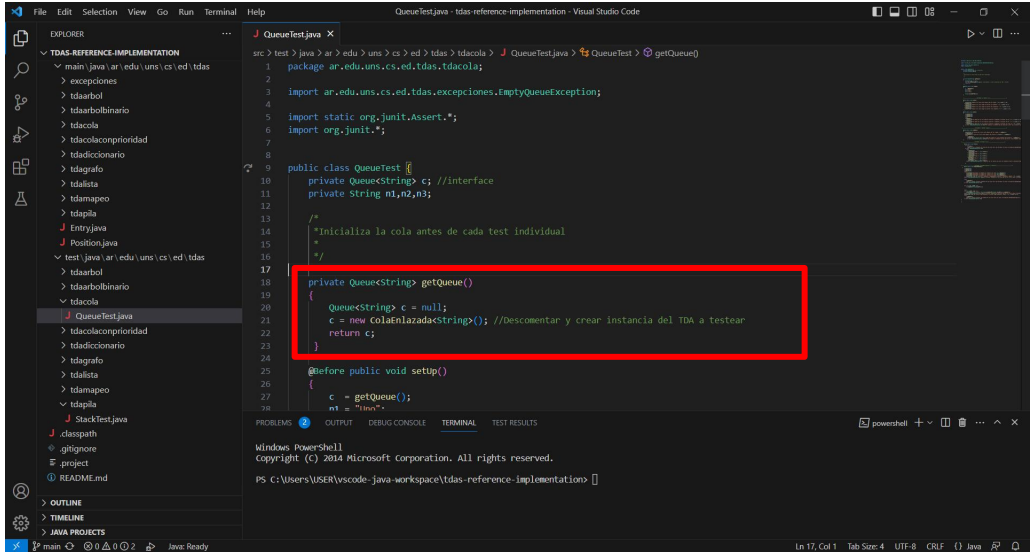
A partir de esto debemos determinar la causa del fallo o error, corregir el código testeado, y/o depurar el test y las llamadas que realiza a nuestro código para encontrar el problema.



Ejecutando JUnit

Si se desea testear alguna otra implementación para el mismo TDA, se puede modificar nuevamente la línea de creación de instancia y ejecutar los tests nuevamente.

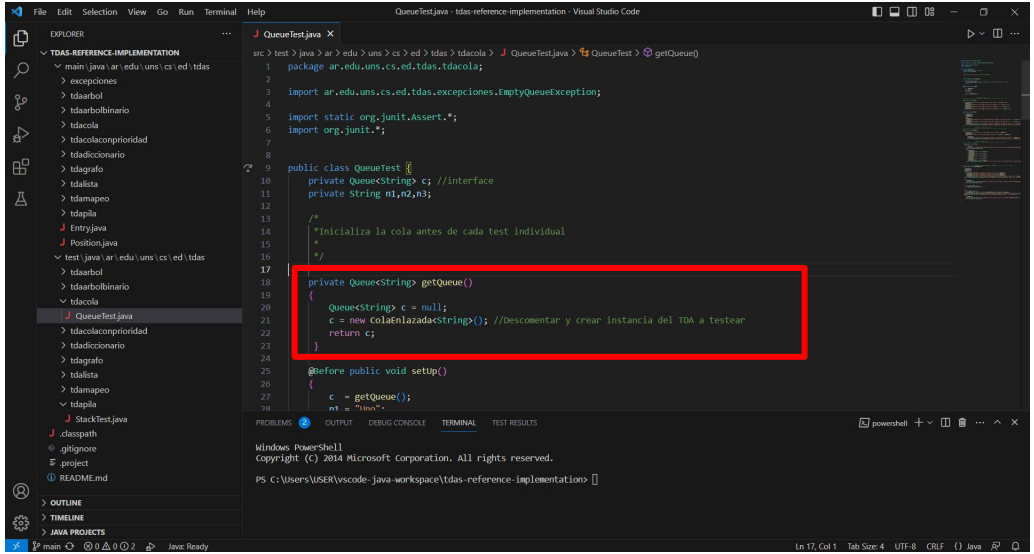
Si se desea preservar el testeo, se puede duplicar el archivo del tester y en la copia reemplazar el objeto a testear, manteniendo de esta manera el testeo para todas las implementaciones.



Ejecutando JUnit

La idea es ir habilitando progresivamente los distintos testers a medida que se van implementando los diferentes TDAs a lo largo de la materia.

Al estar las implementaciones en el mismo paquete que los testers, no hace falta ajustar ninguna otra línea de código ni agregar **import**s.



Bonus: Incorporando JUnit a proyectos de VSCode

Bonus - Incorporando JUnit a proyectos de VSCode

JUnit ya viene pre-configurado en el repositorio de TDAs de la cátedra.

Pero se puede añadir soporte fácilmente en otros proyectos desde Visual Studio Code.

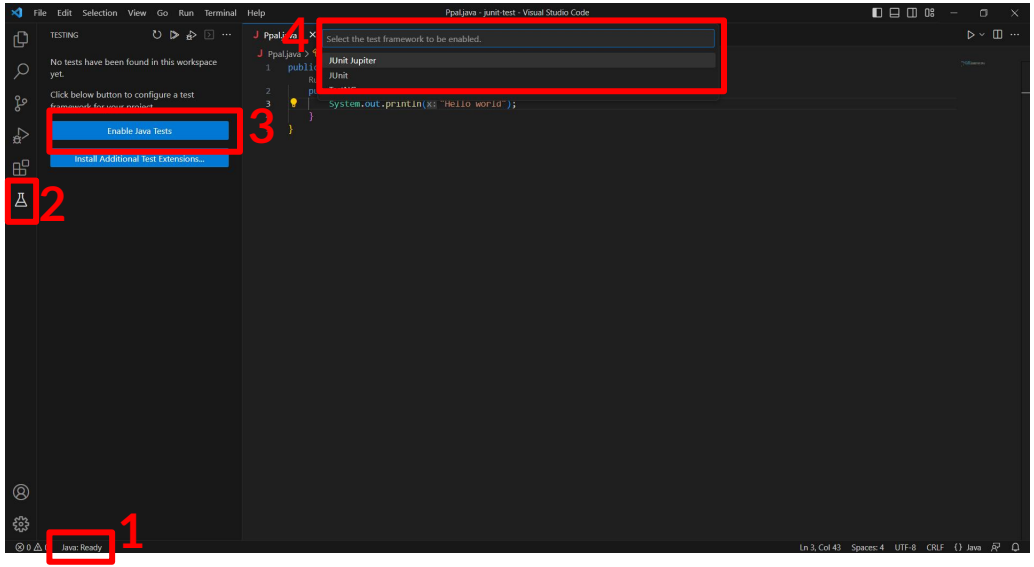
Para usar JUnit se necesita:

- Importar la librería de JUnit correspondiente (ya que no forma parte del lenguaje Java ni de las librerías estándar)
- Hacer los imports correspondientes en los archivos tester
- Utilizar los métodos y anotaciones de la librería para implementar los testers

Bonus - Incorporando JUnit a proyectos de VSCode

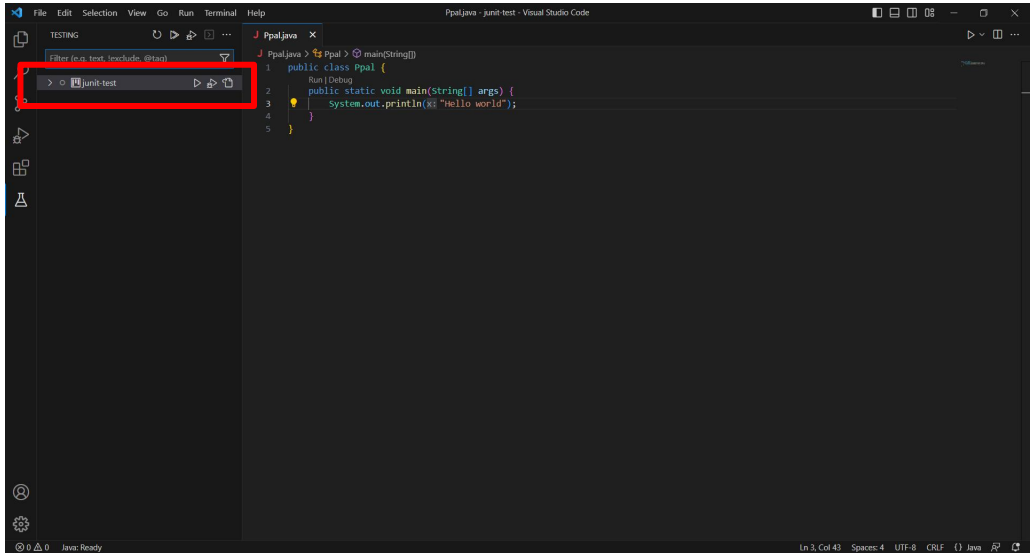
En VSCode se logra:

1. teniendo un proyecto con Java activado (hay que implementar algún archivo .java y lograr que Java se active en el IDE),
2. yendo al panel de **Testing**
3. seleccionando **Enable Java Tests**
4. **JUnit Jupiter** es JUnit 5/6
   JUnit es JUnit 4



Bonus - Incorporando JUnit a proyectos de VSCode

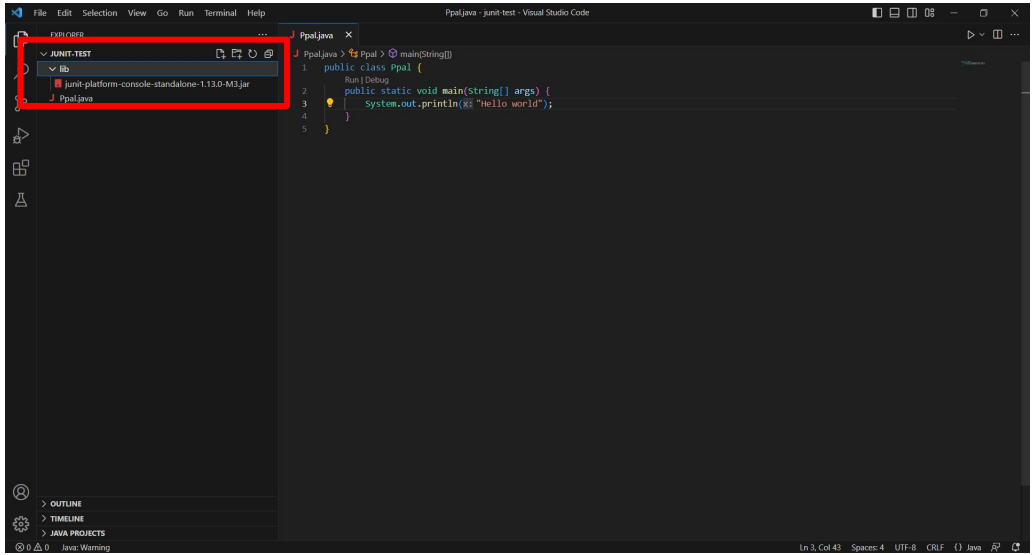
El panel de Testing queda habilitado. Sólo falta programar alguna clase de testing usando JUnit para poder testear.



Bonus - Incorporando JUnit a proyectos de VSCode

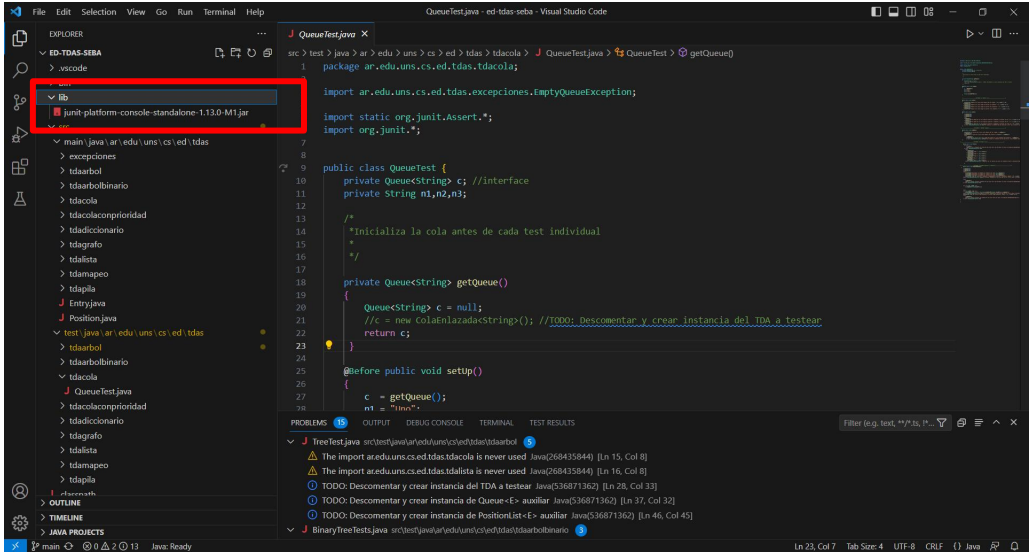
Si observamos los archivos del proyecto, VSCode añadió una carpeta llamada lib y un archivo **.jar** (Java ARchive) que contiene los binarios de JUnit (versión 5 en el caso de este ejemplo).

Cuando se haga referencia a una función de JUnit, el sistema buscará su implementación en estos archivos.



Bonus - Incorporando JUnit a proyectos de VSCode

En el caso del repositorio de los TDAs de la materia, esto ya viene pre-configurado y ya se cuenta con una carpeta **lib** con el .jar de JUnit (JUnit 5 en este caso, que cuenta con compatibilidad hacia atrás para soportar tests de JUnit 4 como los que utilizamos).



Bonus extra: Estructura de un caso de prueba JUnit

Bonus Extra - Estructura de un caso de prueba JUnit

Si bien para testear los TDAs de la materia no es necesario dominar los detalles de implementación de los testers de JUnit, conocerlos puede simplificar entender la lógica que los rige y acelerar la solución de problemas.

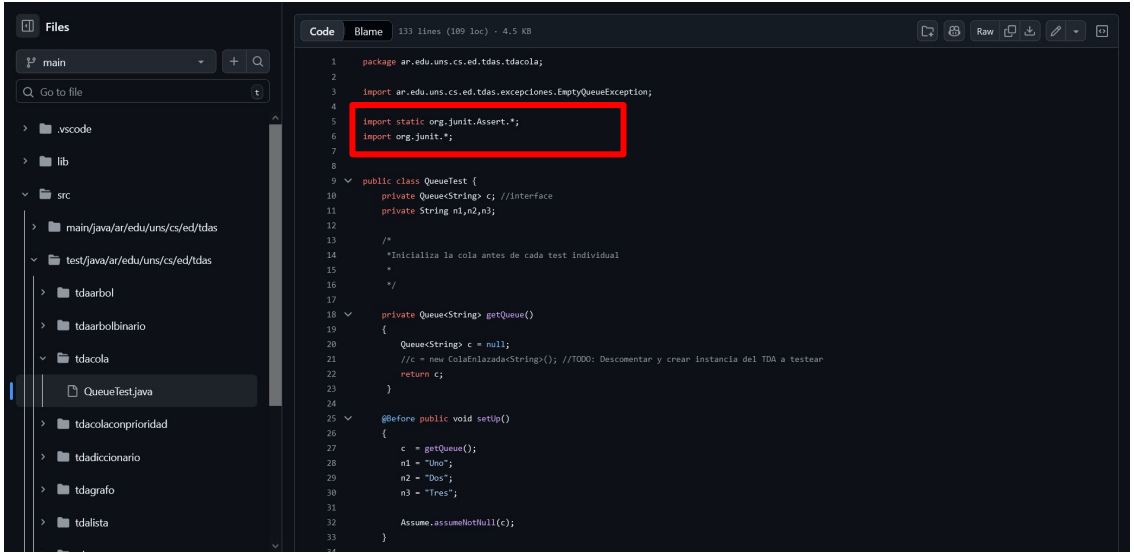
JUnit es un conjunto de librerías que incorpora métodos y anotaciones para determinar cuándo ejecutar ciertas partes de los testers y para definir si la ejecución cumple con ciertas condiciones (llamadas aserciones).

Las anotaciones en Java se escriben con @ y suelen ser utilizadas por herramientas que operan antes del compilador (como JUnit) y que permiten generar código, implementar chequeos, y otras funcionalidades.

Bonus Extra - Estructura de un caso de prueba JUnit

Un conjunto de pruebas JUnit es una clase con ciertas características:

- Importa entidades de **org.junit** y derivados.
- Tiene anotaciones como **@Before**, **@After**, o **@Test** que dotan a los métodos anotados de cierto comportamiento.

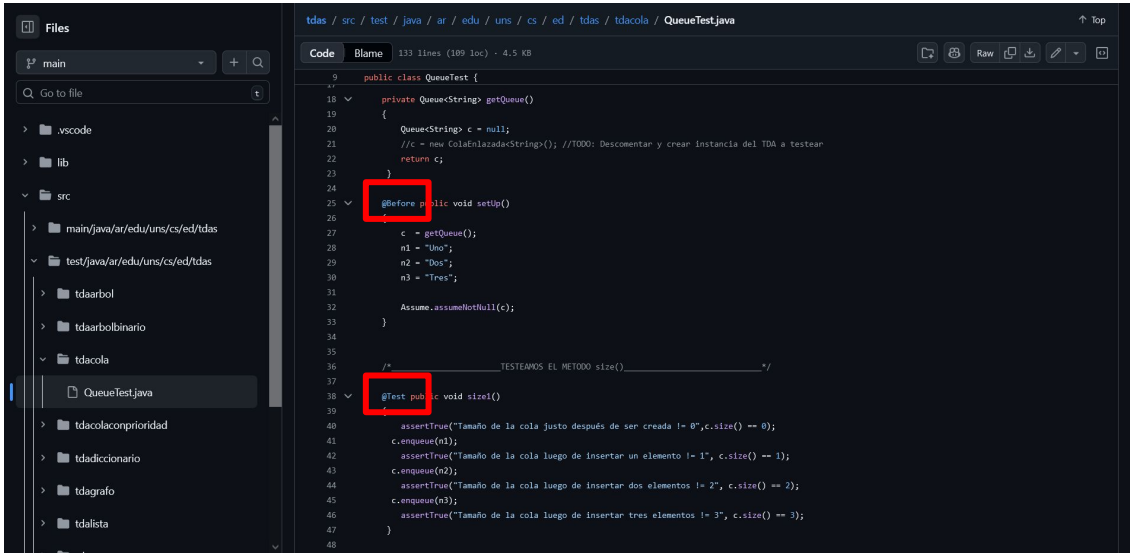


Bonus Extra - Estructura de un caso de prueba JUnit

@Test define un caso de prueba. Los casos de prueba son independientes entre sí y no se ejecutan en orden.

@Before define un método que se ejecuta antes de cada caso de prueba, para inicializar objetos y valores.

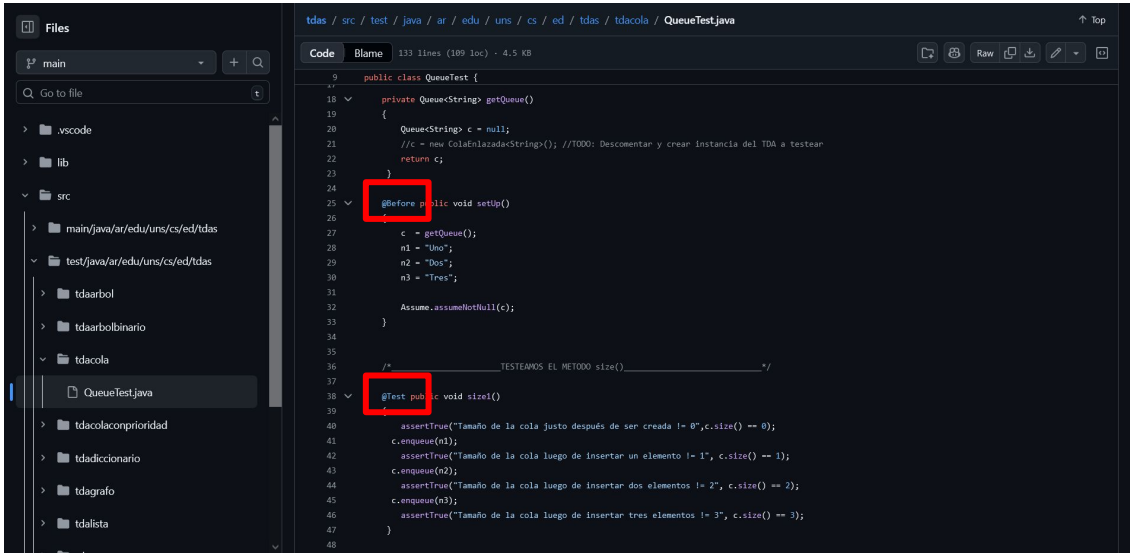
@After define un método que se ejecuta después de cada caso.



Bonus Extra - Estructura de un caso de prueba JUnit

En nuestros tests utilizamos el método anotado con **@Before** para inicializar las estructuras implementadas (de manera directa o mediante un método auxiliar privado que las devuelve).

Antes de ejecutar cada **@Test**, se crea un objeto TDA nuevo para testear (independencia entre tests)

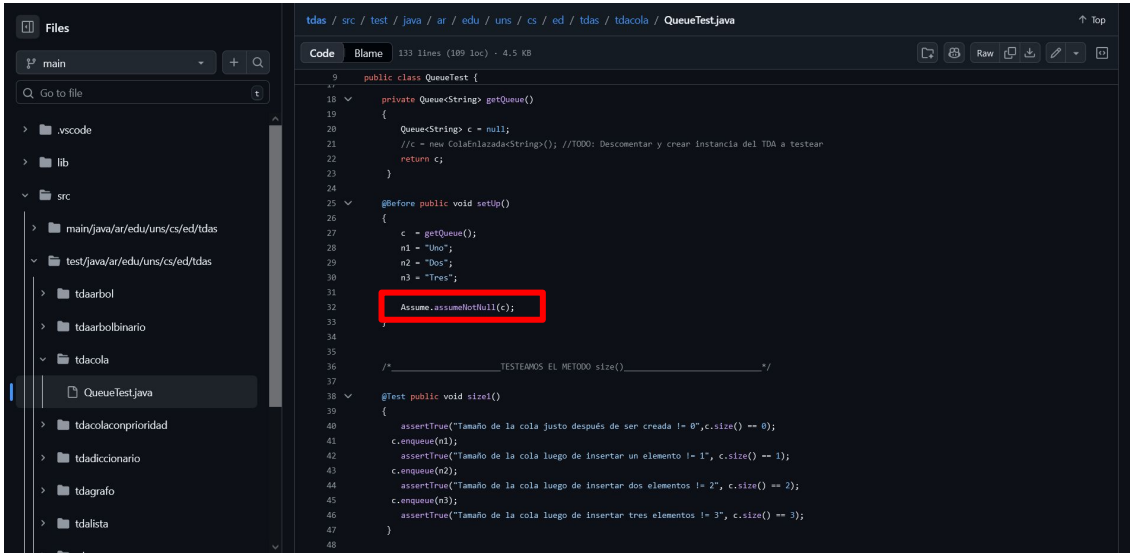


Bonus Extra - Estructura de un caso de prueba JUnit

Dentro de los tests o sus métodos asociados podemos tener Assumptions (suposiciones), por ej. **Assume.assumeNotNull(...)**.

Son condiciones que si no se cumplen saltean el test.

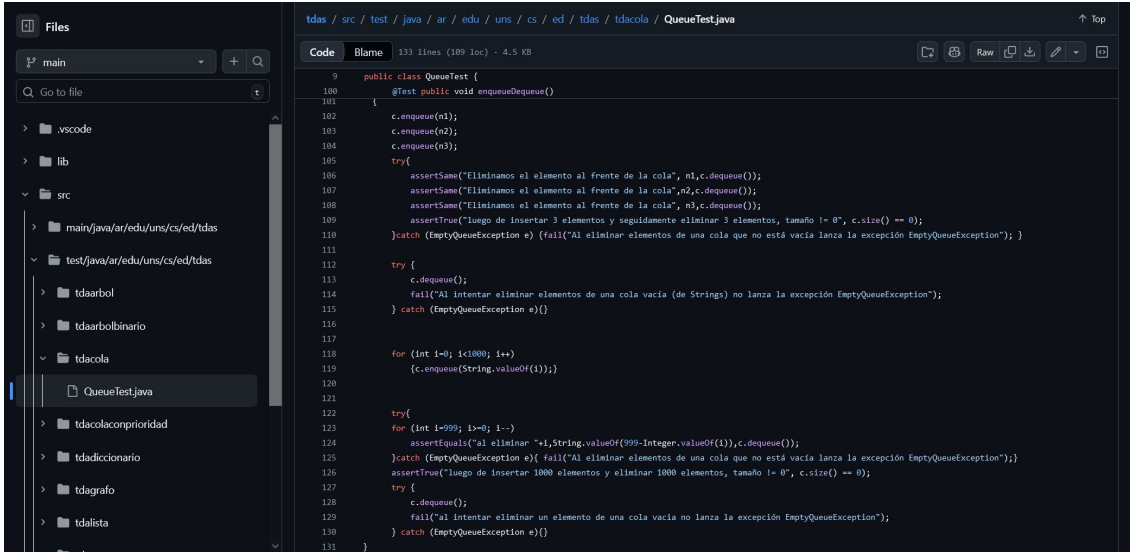
En nuestros testers usamos esto para saltear tests cuyo TDA no está implementado/creado (el/los atributo/s quedan en null)



Bonus Extra - Estructura de un caso de prueba JUnit

Dentro de los tests o sus métodos asociados podemos tener Assertions (aserciones), por ej. **assertTrue, assertFalse, assertSame, assertEquals**, para determinar si una condición es verdadera, falsa o si dos objetos o valores son iguales.

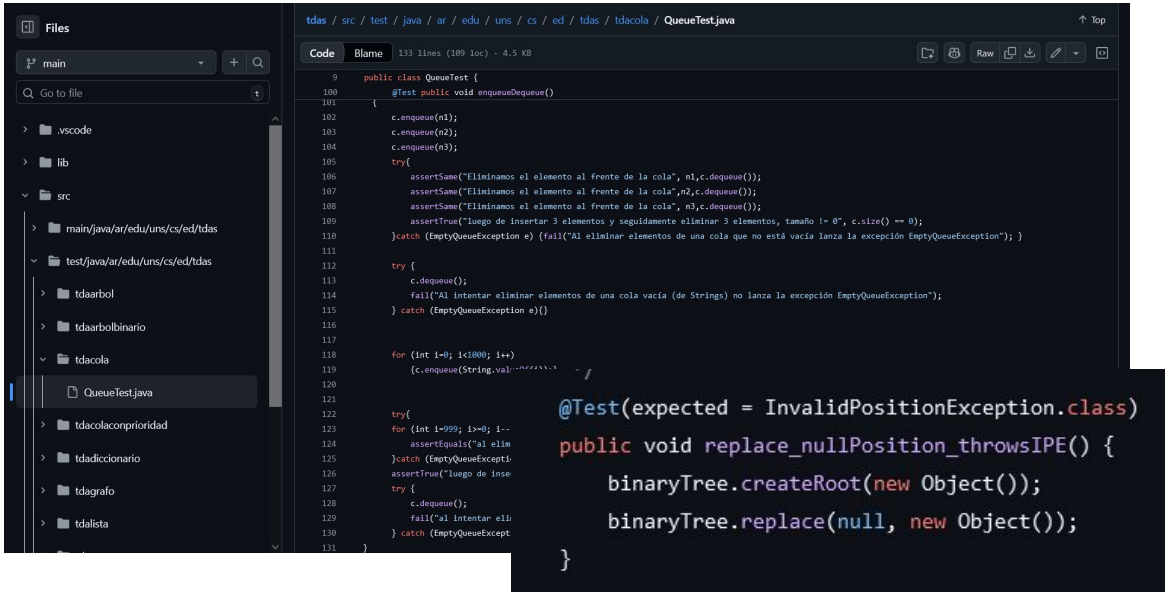
Si alguna condición esperada no se cumple, el test se marca como fallido.



Bonus Extra - Estructura de un caso de prueba JUnit

También tenemos el método **fail** para marcar el fallo del test si se alcanza dicho punto en la ejecución.

O la posibilidad de definir que como resultado de un testeo debería emitirse una excepción particular (anotación **@Test** con **expected** exception) para que el test pase.



Bonus Extra - Estructura de un caso de prueba JUnit

Lo mostrado es sólo una pequeña parte de la funcionalidad de JUnit.

La cantidad de variantes en cuanto a suposiciones y aserciones responde a que el código de los tests sea super claro a la vista respecto de lo que se espera como condiciones válidas. En metodologías como Test Driven Development esto se lleva al extremo y los tests constituyen la documentación del proyecto.

Los casos de prueba de la materia no son tan extremos. Simplemente los utilizamos para verificar que las implementaciones de TDAs realizadas a lo largo del cuatrimestre sean medianamente correctas.

Los tests provistos son mejorables en muchos aspectos pero cumplen muy bien su función.

Bonus Extra - Estructura de un caso de prueba JUnit

JUnit permite especificar casos de prueba que de manera independiente unos de otros verifican los diversos métodos que conforman la interface de los TDAs.

Se intercalan operaciones de creación, modificación y consulta sobre los TDAs con suposiciones y aserciones que definen si los resultados obtenidos son los esperados para cada caso.

La herramienta automatiza la verificación proveyendo un marco de trabajo en el cual con un par de clicks y una rápida revisión de los resultados se puede determinar qué parte de la implementación está fallando (si lo hace) y por qué.

Esto es una primera introducción al Testing Unitario, funcional a las necesidades e Estructuras de Datos. En materias posteriores se avanzará sobre éstos y otros conceptos relacionados.